



VideoPath

API Guide

This guide describes the VideoPath API.

Exported on 12/13/2023



Copyright © Nevion 2023

Document is protected by the NDA agreement and cannot be shared outside without Nevion's approval.

Table of Contents

1	Introduction	6
1.1	Header fields	8
1.2	Subscriptions	9
1.2.1	Adding a new subscription	9
1.2.2	Fetching a subscription change event	10
1.3	RPC API	11
1.3.1	Priority	12
1.3.2	Subscriptions	13
1.4	Authentication	13
1.5	General	14
1.5.1	Time	14
1.5.2	Compression	14
2	API Login	15
2.1	Session Login	15
2.2	Validate Session	16
2.3	Session Logout	16
2.4	Session Less	17
3	Modern Services	18
3.1	Terminology	18
3.2	APIs	18
3.3	Retrieve Service Data	19
3.3.1	Services	19
3.3.1.1	Example Request	19
3.3.1.2	Example Response	19
3.3.1.3	Response Data	20
3.3.2	Profiles	22
3.3.2.1	Example Request	22
3.3.2.2	Example Response	22
3.3.2.3	Response Data	23
3.3.3	Endpoints	23

3.3.3.1 Example Request	23
3.3.3.2 Example Response	23
3.3.3.3 Response Data.....	24
3.4 Synchronous Modern Services	25
3.4.1 General Request Structure	25
3.4.1.1 Request Format.....	25
3.4.1.2 Response Format	27
3.4.2 Set Modern Services.....	31
3.4.2.1 Entry Format	31
3.4.2.2 Create Connection	33
3.4.3 Update Modern Services.....	36
3.4.3.1 Entry Format	36
3.4.3.2 Update Connection	37
3.4.4 Cancel Modern Services.....	40
3.4.4.1 Entry Format	40
3.4.4.2 Cancel Connection	40
3.4.5 Full Example.....	42
3.4.5.1 Create two connections (1):.....	42
3.4.5.2 Locking one of the connections (2):	49
3.4.5.3 Cancelling both connections with best effort (3):	50
3.4.5.4 Unlocking the last connection (4):	52
3.4.5.5 Cancelling the last connection (5):.....	53
3.5 Asynchronous Modern Services	55
3.5.1 Create connection.....	55
3.5.2 Request Status	58
3.5.3 Cancel Connection	60
3.6 Group connection API	61
3.6.1 Retrieve Group Resources	61
3.6.1.1 Request Example	61
3.6.1.2 Response Example	62
3.6.1.3 Response Fields.....	62
3.6.2 Connect Group Resources	63
3.6.2.1 Request Example	63
3.6.2.2 Request Data Example.....	63
3.6.2.3 Request Fields	64

3.6.2.4 Response Example	65
3.6.2.5 Response Fields.....	65
3.6.3 Retrieve Group Connections	65
3.6.3.1 Request Example	66
3.6.3.2 Response Example	66
3.6.3.3 Response Fields.....	67
3.6.4 Update Group Connections.....	68
3.6.4.1 Request Data Example.....	68
3.6.5 Cancel Group Connections	69
3.6.5.1 Request Data Example.....	69
4 Federation	70
4.1 External endpoints.....	70
4.1.1 Example Request	70
4.1.2 Example Response	70
4.1.3 Response Data.....	71
4.2 Handoff points	72
4.2.1 Example Request	72
4.2.2 Example Response	72
4.2.3 Response Data.....	72
4.3 Connections	73

**i Disclaimer**

Nevion reserves the right to make changes without notice to this document. The information provided in this document is for guidance purposes only and shall not form part of any contract © 2023, Nevion. All rights reserved.

Revision	Date	Description
1	2018-11-27	Migrated API documentation from old structure
2	2018-12-06	Updated Modern Services API
3	2019-01-07	Added examples to Modern Services API
4	2019-04-03	Added new partial service concept to Modern Services API suppressConfig field is replaced by allocationState in Modern Services API
5	2020-08-31	Added new async API (8.18)
6	2021-01-25	Restructured pages
7	2021-08-26	Added virtual resource page (group connections)
8	2021-09-09	Added federation section

This documentation describes the VideoPath API. The documentation is divided into different modules as shown below.

This API guide only covers a subset of the VideoPath API that has been released for external use. Please contact Nevion Support if specific data or operations not included in the current API is required for intergration with external systems.

i Note that the Modern Services API is recommended for connection management purposes. The Legacy Services API is still supported for existing integrations, but not recommended for new integrations.



1 Introduction

The VideolPath API consists of a REST-based API and a RPC based API. The REST API is data-driven and allows the user to navigate all data models exported by the VideolPath system using GET requests. Support for POST, PUT, PATCH and DELETE verbs are planned to be released.

The RPC API is function-oriented and allows the user to retrieve and update data in JSON format. Currently, it is necessary to use a combination of the REST and RPC APIs to integrate with the system.

Note that all examples in this API documentation assume that <http://videopath> points to the VideolPath server that we are integrating with.

REST API

The structure of the REST API may be explored directly from a web browser such as Chrome or using an extension like Postman. The base URL for the REST API is:

```
http://videopath/rest/v1/data/
```

To explore the API make sure you are logged into the VideolPath system using an admin account and enter the base URL in the browser followed by a wildcard * at the end to return all children.

```
http://videopath/rest/v1/data/*
```

This returns the following top-level structure in a data structure. Note all data returned from the REST interface is included in the data structure, in addition there are header fields at the same level as the data structure as described below. The header fields contains information about the request.

```
{
  "data": {
    "config": {},          // Configuration data that can be manipulated with various
                           // HTTP verbs. Data below this node changes rarely.
    "status": {},         // Read-only status information. Data below this node is
                           // frequently updated and is a good candidate for subscriptions.
    "experimental": {}    // Experimental data that is not yet documented and contents
                           // may be subject
  }
}
```

To retrieve the next level in the structure the following URL may be used:

```
http://videopath/rest/v1/data/*/*
```

This returns the following structure and the sub-trees that are currently documented here are indicated with comments below:

```
{
  "data": {
    "config": {
      "netservman": {},          // Network services configuration information
    }
  }
}
```



```
"network": {}, // Network topology configuration information
including all resources and their properties.
"devman": {}, // Device manager configuration
"system": {}, // System configuration
"netserman": {}, // Network service manager configuration
"alarms": {}, // Alarm configuration
"metrics": {}, // Performance metric configuration
"analytics": {}, // Analytical configuration
"modernServiceTemplates": {}, // Service definition templates
"profiles": {} // Profile configuration
},
"status": {
  "network": {}, // Network topology status information. This
represents actual information from the network which may differ from the
configuration.
  "persistence": {}, // Persistence status information (datastore)
  "licensing": {}, // System license status
  "pathman": {}, // Path manager information (end-to-end routing)
  "system": {}, // General system information
  "systopo": {}, // System topology status information
  "netserman": {}, // Network services status information
  "devman": {}, // Device manager status information
  "alarms": {}, // Alarm and events information including
current and historic records.
  "healthReport": {}, // System health status information
  "redundancy": {}, // Redundancy controller status information
  "metrics": {}, // Performance metric status information, which
allows retrieval of all collected metrics.
  "analytics": {}, // Analytical status information
  "networkAlarms": {}, // Network related alarm status information
  "devices": {} // Status information pertaining to all devices
and controllers present in the inventory
},
"experimental": {
  ... // Not yet documented and officially released
information is presented here.
}
}
```

To retrieve the structure for data/config/network the following URL may be used:

http://videoipath/rest/v1/data/config/network/*

At this point it is also possible to retrieve all nodes below the URL as follows:

http://videoipath/rest/v1/data/config/network/**

Note that this may return massive amounts of data if used at a high level in the tree structure.



1.1 Header fields

Every REST response includes a header and `_meta` structure as shown below. The header structure was introduced in version 6.8 and onwards, while the `_meta` structure has been included since version 5.0. Note that no specific order of the data, header or `_meta` fields should be assumed.

```
{
  "header" : {
    "auth" : true,
    "code" : 0,
    "errorDetails" : [ ],
    "infoDetails" : [ ],
    "details" : "",
    "msg" : "OK",
    "ok" : true,
    "user" : "sysadmin"
  },
  "_meta" : {
    "type" : "mainResult",
    "info" : {
      "auth" : true,
      "code" : 0,
      "details" : "",
      "msg" : "OK",
      "ok" : true,
      "user" : "sysadmin"
    }
  }
}
```

The `_meta` field is deprecated from release 6.8 and new implementations should use the header field, but it is still included for backwards compatibility with existing integrations.

Description of header fields:

- `auth`: boolean indicating if the user is authenticated (if not authenticated no data is returned)
- `code`: this is set to 0 if successful, otherwise an internal error code is set
- `errorDetails`: includes relevant detailed text descriptions of the error (note this is an array structure)
- `infoDetails`: includes relevant detailed text descriptions pertaining to the request (normally this is empty and note this is an array structure)
- `details`: summary of `errorDetails` or `infoDetails` field
- `msg`: human readable message representing the code field above
- `user`: the authenticated user that issued the request (or "anonymous" if not authenticated)

Note that in subsequent examples the headers are not included as they are common for all REST requests.



1.2 Subscriptions

Information under the /data node in the REST API may contain huge volumes of data. To listen to data changes from the server under the /data node, the REST API supports data subscriptions to efficiently export data changes from the server.

Subscriptions are accessed using the private REST session object. When logging in to a VideolPath server, a session object is created on the server. This object maintains and manages all subscriptions that are used for a given session.

i Subscriptions are only applicable to information below the /data node in the REST API. As a consequence, change events and path specifiers always implicitly refer to paths relative to the /rest/v1/data prefix, as shown in examples below.
Also subscriptions only work for session based (cookie) login. It cannot be combined with one-shot (API) logins, since these are stateless.

The subscriptions for a current session is accessed under the following URL:

`http://<VideoIPathAddress>/rest/v1/sessions/me/subsc`

1.2.1 Adding a new subscription

Adding a new subscription is done by issuing a POST request towards the `/rest/v1/sessions/me/subsc` node. The subscription "path" information is embedded in a JSON document in the body of the request. Note that the format of the "path" attribute in the format document follows the URL format used by GET requests.

Example: To create a subscription equivalent to the normal GET request:

`GET /rest/v1/data/status/alarms/current/**`

Use the following subscription POST request:

`POST /rest/v1/sessions/me/subsc`

`Content-Type: application/json`

`{ "path": "/status/alarms/current/**" }`

This gives the following example response, which contains all current alarms in the system:



```
{
  "_id": "1",
  "data": {
    "status": {
      "alarms": {
        "current": {
          "2:1-2-3-4:123": { ... },
          "2:1-2-3-4:122": { ... }
        }
      }
    }
  }
}
```

In response to a subscription posting, the entire data set matching the query is returned. The format of the response corresponds to the format returned by a normal GET request.

Note that in the returned response, there is an `_id` attribute. This value is allocated by the server and should be used in subsequent requests for this subscription.

1.2.2 Fetching a subscription change event

At any time, the client may ask the server for potential subscription changes. To fetch and acknowledge a change, issue a POST request towards the `/ack` node as shown below. Note the subscription ID as returned in the initial response. If data is changed since the last request, the data node is populated as shown in the example below.

POST `/rest/v1/sessions/me/subsc/1/ack`

Content-type: `application/json`

Body: `<empty>`

The example response below contains one alarm that is updated and one that is deleted. The deleted alarm is tagged with a `"_e"` attribute with value `"d"`. If no `"_e"` attribute is present for the alarm then a full refresh of the alarm is implicit.

```
{
  "_id": "1",
  "data": {
    "_e": "u",
    "status": {
      "_e": "u",
      "alarms": {
        "_e": "u",
        "current": {
          "_e": "u",
          "2:1-2-3-4:123": { ... }
          "2:1-2-3-4:122": { "_e": "d" }
        }
      }
    }
  }
}
```



In case there are no changes since last poll, the data node is set to null.

Subscription change events closely follows the data format as reported directly from the /data node.

The most important attribute is the "`_e`" (event) attribute, which indicates the type of event that applies to the object that it is contained in. This may be set to one of the following values:

<code>_e</code> attribute value	Description
e	Empty. Normally not used, but in case it is set, the entire content from this object and below can be safely ignored.
u	Update. The children of this node has been updated. Only updated IDs are present in the object. All existing children that aren't contained in this object should be kept unchanged.
d	This object has been deleted and should be removed from it's parent.
r	Full refresh. The entire object should be replaced by the contents of the object in this event. All current children should be removed and replaced with the children in this object.

Note that the `_e` attribute is not always present. This implicitly indicates a full refresh.

Between two consecutive ack operations, several events may occur internally to update the subscription. In this time window, change events are merged internally. If an item is added and deleted within this window, a "deleted" event will be reported. External code should handle this case and simply ignore these deleted events.

i A subscription will time out after a while (default time is 60 seconds) if it is not accessed (by posting to the /ack) node.
There is also a maximum limit for a number of subscriptions in a session (default is 128).

1.3 RPC API

The RPC API works like an RPC interface that uses HTTP requests with JSON-documents for requests and responses.

The API user is subject to the same security filtering as normal users using the VideolPath UIs.

This means that the functionality and resources that the API user have access to is filtered based on the roles assigned to the API user in the VideolPath system.

The requests and response messages uses the general structure shown below with a header and data field.

```
{  
  "header": {  
    "id": 0,
```



```
    "priority": "normal"
  },
  "data": {}
}
```

The fields contained in the header and data fields are described for each type of request and response pair. The id field is a sequence number for the request made from the client.

All requests from clients to the server are sent as HTTP POST messages to the URL:

`http://veoipath/api`

For test purposes it is possible to invoke the API using the curl command as shown below:

```
curl -X POST --user username:password -H "Content-type: application/json" -d
'{"header":{"id": 0}, "data":{"includePath":false, "limit":5000}}' http://veoipath/
api/getConnections
```

This example retrieves a list of all current connections in the system.

Note that the `Content-type` HTTP header needs to be set to `application/json` for all requests.

1.3.1 Priority

The RPC API allows requests to be issued with different levels of priority. This is specified using the priority field in the RPC API header below. The default priority is "normal".

```
{
  "header": {
    "id": 0,
    "priority": "normal"
  },
  "data": {}
}
```

The following priority values are supported:

- "low" : background requests that does not need any priority
- "relaxed" : background requests that are not time critical but prioritized above "low"
- "normal" : this is the default priority used if none is specified (also includes UI requests)
- "high" : foreground requests

Note that priority may be specified for all RPC calls, but will only have significant impact for certain calls including connections requests that may be queued by the system.



1.3.2 Subscriptions

Some API requests supports subscriptions, which means that it is possible to specify a subscriptionId. This means that all data is retrieved only on the first request, while subsequent requests with the same subscriptionId retrieves only updated data since the last request. This is used to limit the amount of data that is retrieved with every poll for new data from the server.

The subscriptionId is specified in the request header field as shown below:

```
{
  "header": {
    "id": 0,
    "subscriptionId": "2522b780-fc64-4470-9d1c-9e9e1ada82a7"
  },
  "data": {}
}
```

The subscriptionId should be a UUID generated by the client, which is unique per client and when the request was first made (time factor).

1.4 Authentication

The VeoIPath REST and RPC APIs are based on HTTP, but can also be invoked over HTTPS using SSL/TLS, in order to encrypt the communication between client and server.

User authentication is performed either using basic authentication or using cookie-based authentication.

Basic authentication is normally used for machine-to-machine integration and involves setting the “Basic-authentication” HTTP header to a Base64 encoded string with “user:password”.

Base64 does not provide any significant protection against evesdropping, so for external communication the solution is dependent on HTTPS to ensure privacy and prevent interception of passwords.

Example using basic authentication:

```
curl --user username:password -H "Content-Type: application/json" http://videoipath/
rest/v1/data/status/system/*
```

Note that basic authentication is only allowed for users that are defined as API users in the Security app in VeoIPath.

A client that supports cookie-based authentication can POST a message to the following URL to retrieve a session cookie:

- `http://videoipath/cdb/_session`
- with the data `"name=username&password=password"`

This returns a session cookie (AuthSession) that must be included with subsequent requests to the API.

The main advantage of this approach is that the username and password does not have to be passed with every API request.

Example using cookie authentication:



```
curl --cookie-jar cookies -H "Content-Type: application/x-www-form-urlencoded" -d  
"name=username&password=password" -X POST http://videopath/cdb2/_session  
curl --cookie cookies -H "Content-Type: application/json" http://videopath/rest/v1/  
data/status/system/enums/**
```

1.5 General

1.5.1 Time

The API uses UNIX timestamps for all time values, which means number of milliseconds since January 1st 1970 in UTC. Note that the legacy APIs uses number of seconds and not milliseconds. All timestamps are given and reported in UTC and any conversion to local time zone and daylight savings time is the responsibility of the client.

1.5.2 Compression

The requests specified here return large amounts of JSON formatted data. For this reason it is necessary (or at least highly recommended) for performance reasons to use compression.

The API client should indicate that it supports compression using the following header:

Accept-Encoding: gzip, deflate

The VideolPath server will indicate that data is compressed using the following header:

Content-Encoding: gzip



2 API Login

2.1 Session Login

```
POST /api/_session
```

Authenticate and establish a user session. This request must specify a normal user and not an API user. Note that API users can only be used with basic authentication.

The response includes two cookies VipathSession and XSRF-TOKEN. Both must be included in subsequent requests to the API for authentication purposes. VipathSession is passed back as a cookie, while the value of XSRF-TOKEN is passed back in a X-XSRF-TOKEN header. The latter is not required for GET requests.

Example authentication headers to be included in subsequent requests:

```
X-XSRF-TOKEN: fca1bfd9-e760-4dac-b5b0-f8fed718a73e  
Cookie: VipathSession=3cb092c5-d1e3-4767-bdbb-816eeecba868
```

Cookies may be handled in cURL by using the --cookie-jar option with this request and the --cookie option with subsequent requests.

Request body (HTML form):

```
name=user,password=password
```

Curl example:

```
curl --location --request POST 'http://10.69.25.115/api/_session' \  
--header 'Content-Type: application/x-www-form-urlencoded' \  
--cookie-jar cookies \  
--data-urlencode 'name=user' \  
--data-urlencode 'password=password'
```

Response:

```
{  
  "name": "user",  
  "ok": true,  
  "roles": [  
    "admin"  
  ]  
}
```

Response fields:



Property	Description	Type
name	Username	string
ok	Login succeeded (true or false)	boolean
roles	List of roles	array of string
error	Error message (only in case of failure to login)	string

2.2 Validate Session

```
GET /api/_session
```

Validate a user session. This request should be called every minute to validate a user sessions. A user session will timeout after 30 minutes of inactivity (default setting).

The VipathSession cookie must be passed as a cookie to identify the session.

Curl example:

```
curl --location --request GET 'http://10.69.25.115/api/_session' \
--header 'Cookie: VipathSession=3cb092c5-d1e3-4767-bdbb-816eeecba868'
```

Response:

```
{
  "ok": true,
  "userCtx": {
    "name": "sysadmin",
    "roles": [
      "admin"
    ]
  }
}
```

2.3 Session Logout

```
DELETE /api/_session
```

Validate a user session. This request should be called every minute to validate a user sessions. A user session will timeout after 30 minutes of inactivity (default setting).

The VipathSession cookie must be passed as a cookie to identify the session.



Curl example:

```
curl --location --request GET 'http://10.69.25.115/api/_session' \  
--header 'Cookie: VipathSession=3cb092c5-d1e3-4767-bdbb-816eeecba868'
```

Response:

```
{  
  "ok": true,  
  "userCtx": {  
    "name": "sysadmin",  
    "roles": [  
      "admin"  
    ]  
  }  
}
```

2.4 Session Less

Session login is not required if basic authentication is used. In this case credentials (base64 encoded) are passed with every request. Basic authentication may be specified in cURL by using the --user option.

Example basic authentication header:

```
Authorization: Basic YXBpOnBhc3N3b3Jk
```



3 Modern Services

The Modern Services API replaces the Legacy Services API and offers advanced functionality for creating, retrieving and modifying services within the system.

The Modern Services API is designed to handle many different kinds of services. It does so by separating what is service type specific from what is general for all service types. For example, schedule information is the same regardless of whether you are booking a connection or maintenance, however, a profile id only makes sense in the context of a connection.

3.1 Terminology

- **Allocated booking:** A booking becomes allocated once it has passed validation and has computed a result, e.g. a path for a connection.
- **Booking:** The complete representation of a specific service with schedule information as well as service type specific information.
- **Booking ID:** A unique id given to the booking at create time and will never change.
- **Booking instance/instanced booking:** A booking scheduled once from a specific start time to a specific end time.
- **Endpoint:** A topology vertex that is marked with use as endpoint.
- **Locked:** A user has locked the service. Only this user or an admin user can unlock it. Unless the service is unlocked, modifications will be prevented.
- **Revision:** Every time a specific booking is changed it gets a new revision represented as a string member 'rev'.
- **Recurrent booking:** A recurrent booking is a recurrent generator of normal "one time, start-end" bookings. Instead of having a specific start and end time, we specify a pattern which describes at which recurring times a normal booking should be generated. E.g. daily from 12-16 from next Friday until two weeks. At 11 Friday there will be only one recurrent booking registered, but at 12-16 Friday there will be both the recurrent and the active *instance* registered, unless this is the last instance of the recurrent booking.
- **Recurrent instance:** An instanced booking automatically created by a recurrent booking. No notable differences from a standard instanced bookings except the id and service id will be suffixed with #n, where n is the number of bookings generated by the recurrent booking so far.

3.2 APIs

All service types are set via the same POST /api/... routes:

- /api/setModernServices
- /api/updateModernServices
- /api/cancelModernServices
- /api/modernServicesAsync

For short we will refer to these APIs as "get", "set", "update", "cancel", and "async", respectively.

It is intended that a client either uses set+update+cancel *or* async+update (async also supports cancelling). In practice consumers of the async API rarely needs to use the update API, as more often than not, editing a service in the context of the client results in a new connection in VIP.

The Modern Services API may be used in combination with the following GET /rest API. The get API should be used to read/update state independent of which API is used to change the state. Parsing results of booking updates should primarily be necessary in the event of failures, after which you would get no state update from the get API.

- /rest/v1/data/status/pathman/currentModernServices



3.3 Retrieve Service Data

3.3.1 Services

```
GET /rest/v1/data/status/pathman/currentModernServices/*/type,booking/**
GET /rest/v1/data/status/pathman/currentModernServices/b13fd4a9-454d-4886-bd71-cc2bba1c4f5f/type,booking/**
```

The first request returns all services currently in the system that are accessible to the user. The request includes a filter for the type of service and associated booking data.

The second request returns the type and associated booking data for a specific service with ID `b13fd4a9-454d-4886-bd71-cc2bba1c4f5f`.

3.3.1.1 Example Request

```
curl --request GET \
  --user api:password \
  --url 'http://videoipath/rest/v1/data/status/pathman/currentModernServices/*/type,booking/**'
```

3.3.1.2 Example Response

```
{
  "data" : {
    "status" : {
      "pathman" : {
        "currentModernServices" : {
          "b13fd4a9-454d-4886-bd71-cc2bba1c4f5f" : {
            "type" : "p2mp-join",
            "booking" : {
              "lastModifiedBy" : "sysadmin",
              "parent" : null,
              "templateLink" : null,
              "timestamp" : 1543448395581,
              "tags" : [ "test4" ],
              "descriptor" : {
                "desc" : "test4",
                "label" : "HMBG022/5403_7210-1158_V_13 => Multipoint"
              },
              "allocationState": 0,
              "auditHistory" : [ {
```



```
      "msg" : "New P2MP joined connection created (SRC: virtual.137.2, DST: virtual.136.2, ID: 100649-1). Connection is active from ${timestamp:1543552560} to ${timestamp:1543562340}",
      "user" : "sysadmin",
      "rev" : "",
      "ts" : 1543448395
    } ],
    "groupOrder" : null,
    "to" : "virtual.136.2",
    "rev" : "2-2018-11-29T00:39:55.581+01:00",
    "id" : "b13fd4a9-454d-4886-bd71-cc2bba1c4f5f",
    "profile" : "33c4fe01-899e-43a0-8bd6-ccf67756aff4",
    "from" : "virtual.137.2",
    "createdBy" : "sysadmin",
    "lockedBy" : null,
    "end" : 1543562340000,
    "cancelTime" : null,
    "start" : 1543552560000,
    "serviceId" : "100649-1",
    "isRecurrentInstance" : false
  }
},
...
```

3.3.1.3 Response Data

Field	Type	Values
type	string	Possible values “p2p” (point-to-point) or “p2mp” (point-to-multipoint)
booking	object	
lastModifiedBy	string	User that last modified the service
parent	string	Reference to ID of parent service, null if no parent
templateLink	string	Reference to ID of service template, null if no template
timestamp	number	Timestamp (ms) when the service was last modified
tags	array of string	Array of tags associated with the service
descriptor	object	



Field	Type	Values
desc	string	Description of service
label	string	Label (name) of service
allocationState	int (enum)	0 = normal, 1 = suppressConfig, 2 = noReservation (planned)
auditHistory	array of object	
msg	string	Human readable audit message
user	string	User that initiated change
rev	string	Revision that was changed
ts	number	Timestamp (ms) change was done
groupOrder	number	Services may have a specific order in a group, if no order then null
to	string	ID for the destination endpoint
rev	string	Current revision
id	string	Internal ID for the service
profile	string	ID for the associated profile
from	string	ID for the source endpoint
createdBy	string	Created by specified user
lockedBy	string	Locked by specified user
end	string	Timestamp (ms) for the end time
cancelTime	string	Timestamp (ms) for the cancel time
start	string	Timestamp (ms) for the start time



Field	Type	Values
serviceId	string	Human readable ID associated with the service
isRecurrentInstance	boolean	True if this is an instance of a recurrent booking, otherwise false

Request URL to retrieve all Modern Services in the system (use with caution as it may return large amounts of data):

```
http://videoipath/rest/v1/data/status/pathman/currentModernServices/**
```

Request to retrieve a specific Modern Service:

```
http://videoipath/rest/v1/data/status/pathman/currentModernServices/  
b13fd4a9-454d-4886-bd71-cc2bba1c4f5f/**
```

3.3.2 Profiles

```
GET /rest/v1/data/config/profiles/*/id,name,description,tags/**
```

Get list of profiles available in the system filtered by the ID, name, description and tags associated with the service.

3.3.2.1 Example Request

```
curl --request GET \  
  --user api:password \  
  --url 'http://videoipath/rest/v1/data/config/profiles/*/id,name,description,tags/  
**'
```

3.3.2.2 Example Response

```
{  
  "data" : {  
    "config" : {  
      "profiles" : {  
        "b3fb8e16-b9d5-43bf-9c11-23d923889629" : {  
          "description" : "ST2110-30 2ch 0.125ms 48kHz + ST2022-7",  
          "name" : "Audio 2ch -7",  
          "id" : "b3fb8e16-b9d5-43bf-9c11-23d923889629",  
          "tags" : [ "Audio" ]  
        },  
        "33c4fe01-899e-43a0-8bd6-ccf67756aff4" : {  
          "description" : "ST2110-20 HD + 2022-7",  
          "name" : "Audio 2ch -7",  
          "id" : "33c4fe01-899e-43a0-8bd6-ccf67756aff4",  
          "tags" : [ "Audio" ]  
        }  
      }  
    }  
  }  
}
```



```
"name" : "Video HD -7",  
"id" : "33c4fe01-899e-43a0-8bd6-ccf67756aff4",  
"tags" : [ "Video" ]  
},  
...
```

3.3.2.3 Response Data

Field	Type	Values
description	string	Description of the profile
name	string	Label (name) of the profile
id	string	Internal ID of the profile
tags	array of string	Tags associated with the profile

3.3.3 Endpoints

```
GET /rest/v1/data/config/network/nGraphElements/* where value.useAsEndpoint=true/  
value/useAsEndpoint,codecFormat,vertexType,type,descriptor/desc,label/**
```

Get list of endpoints available in the system that is available to the user. The data is filtered by the ID, name, description, tags and type/equipment properties associated with the endpoint.

3.3.3.1 Example Request

```
curl --request GET \  
  --user api:password \  
  --url 'http://videoipath/rest/v1/data/config/network/nGraphElements/* where  
value.useAsEndpoint=true/value/useAsEndpoint,codecFormat,vertexType,type,descriptor/  
desc,label/**'
```

3.3.3.2 Example Response

```
{  
  "data" : {  
    "config" : {  
      "network" : {  
        "nGraphElements" : {  
          "10.10.10.10" : {
```



```
"value" : {
  "descriptor" : {
    "desc" : "",
    "label" : ""
  },
  "type" : "genericVertex",
  "vertexType" : "Out",
  "useAsEndpoint" : true
},
"device0.3.1000600" : {
  "value" : {
    "descriptor" : {
      "desc" : "",
      "label" : ""
    },
    "codecFormat" : "Video",
    "type" : "codecVertex",
    "vertexType" : "Out",
    "useAsEndpoint" : true
  }
},
...
```

3.3.3.3 Response Data

Field	Type	Values
descriptor	object	
desc	string	Description of the endpoint
label	string	Label (name) of the endpoint
type	string	type of vertex. Typically generic or codecVertex for an endpoint. Remaining fields may vary depending on type.
vertexType	string	Direction of the vertex. In/Out/Bidirectional/Internal.
codecFormat	string	Possible values "Video", "Audio" or "ASI". Only present for codec vertices.
vertexType	string	Possible values "In", "Out" or "BiDirectional"
useAsEndpoint	boolean	True if this vertex is used as endpoint



Field	Type	Values
tags	array of string	Tags associated with the endpoint

3.4 Synchronous Modern Services

This section includes documentation for the following POST /api/... routes:

```
POST /api/setModernServices
POST /api/updateModernServices
POST /api/cancelModernServices
```

The set/update/cancel APIs specified here are synchronous in the sense that they will not return a response until the result of the booking request is available. This means that all resources required by the service is either allocated or de-allocated, but does not necessarily guarantee that all resources have been configured upon return.

3.4.1 General Request Structure

3.4.1.1 Request Format

All Synchronous Modern Services APIs share the same POST data structure:

```
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,
    "bookingStrategy": 2,
    "entries": [{entry1}, {entry2}, ...]
  }
}
```

The header and data outer-most structure is common for all POST /api/... routes and not documented here.

The content of the data structure is as follows:

Field	Type	Values
entries	array of object	Object schema depends on which API is called. See <i>Entries for 'set'</i> , <i>Entries for 'update'</i> , and <i>Entries for 'cancel'</i> below.
conflictStrategy	int	See the <i>Conflict strategy</i> section below.



Field	Type	Values
bookingStrategy	int	See the <i>Booking strategy</i> section below.

Booking Strategy

This parameter controls how the backend handles the booking of the given entries.

Int value	Label (supported version)	Description
0	All or nothing	Unless all requested changes succeed, don't alter the backend state.
1	Validate only	Dry run. Backend state is not altered, but response and validation is performed as if it was a proper change request.
2	Best effort (7.10+)	Only entries that succeed will be altered in the backend state. If we request to set three services and two fail due to e.g. maxed capacity, only one service will be present backend. If this option is used, the order of the entries should reflect their priority (most important first).
3	Best effort iterative (7.12+)	Recursively attempts to book the given set of services. More resource intensive than best effort in the event of failures, but produces a better end state in the event of failures and non trivial routing rules.
4	Allow invalid (8.10+)	Even if a booking fails it will be allowed into the registry as an invalid booking. It will appear in the get API on rest and will have a booking alert associated with it.
5	Bulk routing (8.18+)	Given that the reachability strategy is set to leaf/spine (i.e. fast routing) we perform site-site routing in bulk since the inter-site paths are deterministic.

Booking strategy was introduced in version 7.10, however, in previous versions "validateOnly=true" was supported (now deprecated).

For backwards compatibility this boolean is still supported, and will be automatically substituted with "bookingStrategy=1" if present.



Conflict Strategy

If we get an endpoint conflict, e.g. destination already in use, the request typically fails with a resource error status. However, if a conflict strategy is provided, the backend can automatically attempt to resolve such conflicts.

Int val ue	Label	Description
0	No strateg y	No attempt to resolve conflicts will be made. This is the default behaviour.
1	Cancel destina tion	If we get a destination conflict, the conflicting bookings will be attempted cancelled in the same request.
2	Un- allocat e source and destina tion	If we get a destination and/or a source conflict, the conflicting bookings will be unallocated, i.e. their resources are freed up. See allocationState in connection service definitions for more details regarding this.

Conflict strategy was introduced in version 7.10.

3.4.1.2 Response Format

All APIs share response structure. If the incoming request validation fails, a HTTP error is provided by standard means (failure status code).

If the request is accepted and handled by the backend, error and success reporting is done in a custom fashion as described below:

```
{
  "header": {
    "msg": [
      ""
    ],
    "caption": "Operation Successful",
    "status": "OK",
    "ok": true,
    "id": 1
  },
  "data": { ... }
}
```



The first thing to check is the header field. If the field "ok" is false, then the data object is empty and the causes are listed in the "msg" field. For the best effort booking strategy this field is always true so header checking can be skipped.

If the header is ok, then we proceed into the data object. If we requested to set or update three bookings we get a data object like this:

```
{
  "entriesLink": [
    {
      "error": null,
      "id": "1423654-2"
    },
    {
      "error": null,
      "id": "1423656-1"
    },
    {
      "error": "Some error message",
      "id": null
    }
  ],
  "bookresult": {
    "result": {
      "msg": [
        "Booking failed with 1 source conflict and 1 destination conflict"
      ],
      "ok": false
    },
    "details": {
      "1423656-1": {
        "rev": "2-2019-05-07T07:46:46.758+02:00",
        "status": 0,
        "type": "connection",
        ... more type specific fields
      },
      "1423654-2": {
        "rev": "2-2019-05-07T07:46:46.724+02:00",
        "status": -7,
        "type": "connection",
        ... more type specific fields
      }
    }
  }
}
```

Description of response fields:



Field	Type	Values
entries Link	array of object	First element represents result of first entry in the "entries" field of the request data, second element represents the second entry, and so on. If the id field is defined, a detailed report is available for this entry in the bookresult.details map using this id as key. If the id field is not defined, then an error occurred in the entries validation, hence an id may not have been generated or the id can not be trusted (id is null in the response in this case). An error message describing this should be available as a string in the "error" field. If the data object is defined it should always be possible to zip the request entries with the response entriesLink, for then to iterate over request and response tuples.
bookresult	object	
result	object	
msg	array of strings	An overall message representing the outcome of the request.
ok	boolean	A convenient accumulated value that is false if at least one of the requested entries failed.
details	object	map from booking id to a booking result object. Each service type has its own result object, but all result objects has a status code and holds the latest revision of the booking for this id. Status codes can be read from /rest/v1/data/status/system/enums/bookingStatusCode.

Status Codes

In general negative status codes are direct errors, and other status codes are successes. However, it is up to the client's response handler how to interpret this.

If we e.g. request to cancel a booking with an id that doesn't exist (e.g. already cancelled), then it is not a failure since the backend state after the request is that *that* booking id is removed.

If the api user wants a stricter error handling, then all non-zero values should be regarded as errors.

The list of status codes is:

```
{
  "0": "Booking OK"
  "4": "Cancel ignored not found"
  "8": "Booking already OK"
  "-1": "No path found"
  "-2": "Destination endpoint not found"
  "-3": "Source endpoint not found"
```



```
"-4": "Source and destination not found"
"-7": "Resource conflict"
"-8": "Illegal data"
"-9": "Out of date"
"-10": "Unexpected error"
"-11": "No route"
"-12": "Capacity exceeded"
"-13": "Format mismatch"
"-15": "Illegal recurrence"
"-16": "Prohibited path change"
"-17": "Booking not found"
"-18": "Epp vertex not found"
"-19": "Requested VLAN is unavailable"
"-20": "VLAN range exhausted"
"-21": "Inherited VLAN is illegal"
"-22": "Required edge not found"
"-24": "Profile not found"
"-25": "Partial product"
"-26": "Locked"
"-27": "Unauthorized"
"-28": "SIPS split-merge points not found"
"-29": "License depleted"
"-30": "Multicast routing conflict"
"-31": "External error"
}
```

Validation

There are four layers of validation.

Incoming request validation:

First barrier validation like authentication, raw json validation against the backend data models. If the request is unauthorized or required fields are missing etc. the request will fail with an HTTP error.

Entries validation:

At this stage the json data has been successfully deserialized, and we can validate the contents of the data in the context of the backend state.

Failures at this stage is typically requesting to update a booking with an id that is not present, or updating a connection service with something else than a connection update definition.

Booking validation:

At this stage a number of standard checks are performed. A request will fail if...

- An instanced finite booking duration is more than a year
- Start is before end
- A recurrence pattern with zero active instances is requested
- The revision provided in update/cancel is wrong
- A child of partial services is attempted to be updated/cancelled by a user
- A locked booking is attempted to be updated/cancelled
- A locked booking is attempted to be unlocked by a user with no such permissions



- ... service type specific validations (see respective service type section)

Result calculation:

A result is attempted to be calculated, e.g. a path for a connection booking. If path finding succeeds we are able to allocate the booking and succeed the request entry. If no such path is available, a result is generated with detailed information about the failure.

3.4.2 Set Modern Services

```
POST /api/setModernServices
```

The set modern services API is also referred to as the set API. It is typically combined with the update API (/api/updateModernServices) and cancel API (/api/cancelModernServices).

The purpose of the set API is to book a new service in the system with the possibility to specify scheduling options.

The API may be used to define different type of services including connection, group and partial services.

3.4.2.1 Entry Format

Please refer to the section [General Request Structure](#) (see page 25) for the overall structure of the set request. The entry format below fits into the general structure and is specific for the set request.

```
{
  "scheduleInfo": { ... },
  "locked": false,
  "serviceDefinition": {
    "type": "connection",
    "descriptor": {
      "desc": "desc",
      "label": "label"
    },
    "tags": [
      "tag"
    ],
    ... type specific fields
  }
}
```

The entry structure for the set API includes the following fields:

Field	Type	Values
scheduleInfo	object	See below.
locked	boolean	Initial locked status of the service.



Field	Type	Values
serviceDefinition	object	type-dependent except for descriptor and tags which are as described in the Service Data section.

Schedule Information

The scheduleInfo object is equivalent to the scheduleInfo from the Service Data section, however, some fields which are virtual fields generated for human readability are not expected on input, and an additional type tag is needed to distinguish a recurring from a non-recurring schedule.

Normal non-recurrent bookings ("once"):

Field	Type	Values
type	string	"once"
startTimestamp	long	Timestamp (ms) for the start time
endTimestamp	long	Timestamp (ms) for the end time

Recurrent bookings ("recurring"):

Field	Type	Values
type	string	"recurring"
pattern		
pattern	int	0=daily, 1=weekly, 2=monthly
startTime	long	Start timestamp (ms) for recurrence pattern. Should correspond to a 00:00 time in the given time zone.
endTime	long	End timestamp for recurrence pattern. The last date a pattern is in use. Any time within the day will suffice.
timeZoneId	string	A time zone spec. for this pattern. Full IANA names are required; e.g. "Europe/London"
iterationFilter	array of long	Timestamps (ms) of exceptions to the recurrence pattern. Any timestamp during the day will do.
weekDays	array of int	Set of days to include when using weekly pattern; 1=Monday, 2=Tuesday etc.



Field	Type	Values
instance		
localStartTime	long	Start time within the day, measured in seconds from 00:00.
localEndTime	long	End time within the day, measured in seconds from 00:00.

The type of the schedule info is used to determine whether a recurrent or an instanced booking should be generated. Service definitions do **not** depend on the schedule in any way.

3.4.2.2 Create Connection

This section describes how to create one or more connections using the set API.

The following type specific fields are used to create a new connection in the system.

```
{
  "to": "device0.3.1000600",
  "from": "device1.3.90600",
  "allocationState": 0,
  "profileId": "10355806-c553-4821-b7e7-f8fd81a1826c",
  "ctype": 2,
  "skipReuseJoinId": false,
  ... generic fields
}
```

Create new connection field descriptions:

Field	Type	Values	Field alias (version)
to	string	vertex id of the destination/sink/receiver endpoint	dests.destId (7.10)
from	string	vertex id of the source/transmitter endpoint	srcId (7.10)
allocationState	int	See below.	
profileId	string	ID of the profile associated with the service.	



Field	Type	Values	Field alias (version)
ctype	int	0 = Point to point (unicast) 2 = point to multipoint (multicast) 3 = bidirectional.	routing (7.10)
skipReuseJoinId	Boolean	If true, multicast services will get service ids the same way as unicast (no -1 -2 -3 etc) Recommended for large salvos where setup time is critical.	

Allocation State

Value	Label	Description
0	Normal	Booking is allocated and reserves resources and is configured normally.
1	Suppress config	Booking is allocated and reserves resources but is not configured.
2	No reservation	Booking is allocated but does not reserve resources and is not configured (planned).

Deprecations and backwards compatibility

- In 7.10 *allocationState* replaced a boolean *suppressConfig=true/false*. If this previous scheme is provided it will automatically be substituted with *allocationState=1*.
- In 7.10 the field *dests* was replaced with *to* in order to unify the service definition with the allocated booking.
- In the previous version it was possible to set multiple *tos* in a single entry since the *dests* was an array. Version 7.10 is not backwards compatible with this pattern and will expect a single entry with N destinations into N entries. This change was motivated by most api consumers using N=1 anyhow.

Request Example

```
curl --request POST \  
  --user api:password \  
  --url 'http://videopath/api/setModernServices' \  
  --header 'Content-Type: application/json' \  
  --data @requestData.json
```



Request Data

Request

```
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,
    "bookingStrategy": 2,
    "entries": [
      {
        "scheduleInfo": {
          "startTimestamp": 1557310613835,
          "type": "once",
          "endTimestamp": 1557310623835
        },
        "locked": false,
        "serviceDefinition": {
          "to": "device0.3.1000600",
          "from": "device1.3.90600",
          "allocationState": 0,
          "descriptor": {
            "desc": "desc",
            "label": "label"
          },
          "profileId": "10355806-c553-4821-b7e7-f8fd81a1826c",
          "tags": [
            "tag"
          ],
          "type": "connection",
          "ctype": 2
        }
      }
    ]
  }
}
```

Response Example

```
{
  "data": {
    "entriesLink": [
      {
        "error": null,
        "id": "1423699-1"
      }
    ]
  }
}
```



```
    ],
    "bookresult": {
      "result": {
        "msg": [],
        "ok": true
      },
      "details": {
        "1423699-1": {
          "availableSpots": [],
          "triedSpotFinding": false,
          "rev": "2-2019-05-08T12:16:50.841+02:00",
          "status": 0,
          "details": [],
          "type": "connection"
        }
      }
    },
    "tmpIdMap": {}
  },
  "header": {
    "caption": "Operation Successful",
    "id": 1,
    "msg": [
      ""
    ],
    "ok": true,
    "status": "OK"
  }
}
```

3.4.3 Update Modern Services

The entry structure for the set API includes the following fields

```
POST /api/updateModernServices
```

The update modern services API is also referred to as the update API. It is typically combined with the set API (/api/setModernServices) and cancel API (/api/cancelModernServices).

The purpose of the set API is to update a service in the system.

The API may be used to define different type of services including connection, group and partial services.

3.4.3.1 Entry Format

Please refer to the section [General Request Structure](#) (see page 25) for the overall structure of the update request. The entry format below fits into the general structure and is specific for the update request.

```
{
  "id": "1423642-1",
  "rev": "2-2019-05-07T06:28:25.136+02:00",
```



```
"serviceParameters": {
  "scheduleInfo": { ... },
  "locked": true
},
"genericParameters": {
  "descriptor": {
    "label": "label",
    "desc": "desc"
  },
  "type": "connection",
  "tags": [
    "tag",
    "new tag"
  ],
  ... type specific fields
}
```

The entry structure for the update request includes the following fields:

Field	Type	Values	Field alias (version)
id	string	Booking id	
rev	string	Booking revision	
genericParameters	object	Optional. Set of parameters independent of the service type.	inheritable (7.10)
locked	boolean	Same as for set.	
scheduleInfo	object	Same as for set.	
serviceParameters	object	Optional. Set of parameters specific to a service type.	updater (7.10)
descriptor	object	Same as for set.	
tags	array of strings	Same as for set.	

At least one of the optional objects should be included. To not include an optional field, set it to "null" or in most cases just skipping the field works as well.

3.4.3.2 Update Connection

This section describes how to create one or more connections using the set API.



The following type specific fields are used to create a new connection in the system.

```
{
  "profileId": null,
  "allocationState": 0,
  ... generic fields
}
```

Update connection field descriptions:

Field	Type	Values
profileId	string (nullable)	Same as for the service definition. If null or not present, the current profile will be used.
allocation State	int	Same as for the service definition.

Deprecations and backwards compatibility:
Same allocationState-suppressConfig translation as for the set service definition.

Request Example

```
curl --request POST \
  --user api:password \
  --url 'http://videoipath/api/updateModernServices' \
  --header 'Content-Type: application/json' \
  --data @requestData.json
```

Request Data

```
Request
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,
    "bookingStrategy": 2,
    "entries": [
      {
        "serviceParameters": {
          "descriptor": {
            "label": "label",

```



```
        "desc": "desc"
      },
      "profileId": null,
      "type": "connection",
      "allocationState": 0,
      "tags": [
        "tag",
        "new tag"
      ]
    },
    "genericParameters": {
      "scheduleInfo": {
        "startTimestamp": 1557310613835,
        "type": "once",
        "endTimestamp": 1557310623835
      },
      "locked": true
    },
    "rev": "2-2019-05-08T12:16:50.841+02:00",
    "id": "1423699-1"
  }
]
}
```

Response Example

```
{
  "data": {
    "entriesLink": [
      {
        "error": null,
        "id": "1423699-1"
      }
    ],
    "bookresult": {
      "result": {
        "msg": [],
        "ok": true
      },
      "details": {
        "1423699-1": {
          "availableSpots": [],
          "triedSpotFinding": false,
          "rev": "3-2019-05-08T12:16:50.941+02:00",
          "status": 0,
          "details": [],
          "type": "connection"
        }
      }
    }
  },
}
```



```
    "tmpIdMap": {}
  },
  "header": {
    "caption": "Operation Successful",
    "id": 1,
    "msg": [
      ""
    ],
    "ok": true,
    "status": "OK"
  }
}
```

3.4.4 Cancel Modern Services

The entry structure for the set API includes the following fields

```
POST /api/cancelModernServices
```

The cancel modern services API is also referred to as the cancel API. It is typically combined with the set API (/api/setModernServices) and update API (/api/updateModernServices).

The purpose of the set API is to cancel a service in the system.

The API may be used to define different type of services including connection, group and partial services.

3.4.4.1 Entry Format

Please refer to the section [General Request Structure](#) (see page 25) for the overall structure of the update request. The entry format below fits into the general structure and is specific for the cancel request.

```
{
  "id": "1423642-1",
  "rev": "2-2019-05-07T06:28:25.136+02:00"
}
```

To cancel a service just pass the id and the revision.

3.4.4.2 Cancel Connection

Request Example

```
curl --request POST \
  --user api:password \
  --url 'http://videoipath/api/cancelModernServices' \
  --header 'Content-Type: application/json' \
  --data @requestData.json
```




Request Data

Cancelling two connections with best effort.

Request

```
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,
    "bookingStrategy": 2,
    "entries": [
      {
        "rev": "2-2019-05-08T12:16:50.843+02:00",
        "id": "1423700-1"
      },
      {
        "rev": "3-2019-05-08T12:16:50.941+02:00",
        "id": "1423699-1"
      }
    ]
  }
}
```

Response Example

Note one cancel request failed while the other succeeded.

```
{
  "data": {
    "entriesLink": [
      {
        "error": null,
        "id": "1423700-1"
      },
      {
        "error": null,
        "id": "1423699-1"
      }
    ],
    "bookresult": {
      "result": {
        "msg": [
          "The service is locked."
        ],
        "ok": false
      },
      "details": {
```



```
{
  "1423699-1": {
    "rev": "3-2019-05-08T12:16:50.941+02:00",
    "status": -26,
    "type": "generic"
  },
  "1423700-1": {
    "availableSpots": [],
    "triedSpotFinding": false,
    "rev": "",
    "status": 0,
    "details": [],
    "type": "connection"
  }
},
"tmpIdMap": {}
},
"header": {
  "caption": "Operation Successful",
  "id": 1,
  "msg": [
    ""
  ],
  "ok": true,
  "status": "OK"
}
}
```

3.4.5 Full Example

This section will demonstrate how to create two connections (1), lock one of them (2), and attempt to cancel both with best effort (3). This will fail since one of them is locked, but since we use best effort the unlocked one should succeed.

We then unlock the last locked connection (4) and cancel it (5):

3.4.5.1 Create two connections (1):

We will connect "device1.3.90600" to "device0.3.1000600" and "device1.3.90602" to "device0.3.1000602". Both use the profile with id "10355806-c553-4821-b7e7-f8fd81a1826c" and multicast (ctype=2), and the second one starts off with suppressed configuration (allocationState=1):

Request

```
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,

```



```
"bookingStrategy": 2,
"entries": [
  {
    "scheduleInfo": {
      "startTimestamp": 1557310613835,
      "type": "once",
      "endTimestamp": 1557310623835
    },
    "locked": false,
    "serviceDefinition": {
      "to": "device0.3.1000600",
      "from": "device1.3.90600",
      "allocationState": 0,
      "descriptor": {
        "desc": "desc",
        "label": "label"
      },
      "profileId": "10355806-c553-4821-b7e7-f8fd81a1826c",
      "tags": [
        "tag"
      ],
      "type": "connection",
      "ctype": 2
    }
  },
  {
    "scheduleInfo": {
      "startTimestamp": 1557310613835,
      "type": "once",
      "endTimestamp": 1557310623835
    },
    "locked": false,
    "serviceDefinition": {
      "to": "device0.3.1000602",
      "from": "device1.3.90602",
      "allocationState": 1,
      "descriptor": {
        "desc": "desc",
        "label": "label"
      },
      "profileId": "10355806-c553-4821-b7e7-f8fd81a1826c",
      "tags": [
        "tag"
      ],
      "type": "connection",
      "ctype": 2
    }
  }
]
}
```

**Response**

```
{
  "data": {
    "entriesLink": [
      {
        "error": null,
        "id": "1423699-1"
      },
      {
        "error": null,
        "id": "1423700-1"
      }
    ],
    "bookresult": {
      "result": {
        "msg": [],
        "ok": true
      },
      "details": {
        "1423700-1": {
          "availableSpots": [],
          "triedSpotFinding": false,
          "rev": "2-2019-05-08T12:16:50.843+02:00",
          "status": 0,
          "details": [],
          "type": "connection"
        },
        "1423699-1": {
          "availableSpots": [],
          "triedSpotFinding": false,
          "rev": "2-2019-05-08T12:16:50.841+02:00",
          "status": 0,
          "details": [],
          "type": "connection"
        }
      }
    },
    "tmpIdMap": {}
  },
  "header": {
    "caption": "Operation Successful",
    "id": 1,
    "msg": [
      ""
    ],
    "ok": true,
    "status": "OK"
  }
}
```

The currentModernServices REST api now reads:



```
{
  "1423699-1": {
    "formatSubState": 0,
    "legacyType": "p2mp-join",
    "needPersist": true,
    "state": 0,
    "res": {
      "paths": [
        {
          "intervals": null,
          "path": {
            "bidirReversePath": false,
            "main": {
              "edges": [
                {
                  "fromNode": "device1.3.90600",
                  "toNode": "device1.3.10604.out",
                  "id": "device1.3.90600::device1.3.10604.out"
                },
                {
                  "fromNode": "device1.3.10604.out",
                  "toNode": "device0.3.10604.in",
                  "id": "5191ac7d-f279-4b9b-a281-eb81be808ee0"
                },
                {
                  "fromNode": "device0.3.10604.in",
                  "toNode": "device0.3.1000600",
                  "id": "device0.3.10604.in::device0.3.1000600"
                }
              ]
            },
            "status": 0
          },
          "spare": {
            "edges": [
              {
                "fromNode": "device1.3.90600",
                "toNode": "device1.3.10606.out",
                "id": "device1.3.90600::device1.3.10606.out"
              },
              {
                "fromNode": "device1.3.10606.out",
                "toNode": "device0.3.10606.in",
                "id": "10dc0597-8d23-4580-b958-d8a3c17f7d85"
              },
              {
                "fromNode": "device0.3.10606.in",
                "toNode": "device0.3.1000600",
                "id": "device0.3.10606.in::device0.3.1000600"
              }
            ],
            "status": 0
          }
        }
      ],
      "status": 0
    }
  }
}
```



```
    }
  ],
  "topLevelStatus": null,
  "type": "bookingResult"
},
"type": "connection",
"resultSubState": 0,
"booking": {
  "lastModifiedBy": "api",
  "parent": null,
  "legacyType": "p2mp-join",
  "templateLink": null,
  "timestamp": 1557310610841,
  "ctype": 2,
  "tags": [
    "tag"
  ],
  "descriptor": {
    "desc": "desc",
    "label": "label"
  },
  "suppressConfig": false,
  "auditHistory": [
    {
      "msg": "New P2MP joined connection created (SRC: device1.3.90600, DST: device0.3.1000600, ID: 1423699-1). Connection is active from ${timestamp:1557310613} to ${timestamp:1557310623}",
      "user": "api",
      "rev": "",
      "ts": 1557310610
    }
  ],
  "groupOrder": null,
  "scheduleInfo": {
    "startTimestamp": 1557310613835,
    "endTimestamp": 1557310623835,
    "end": "2019-05-08T12:17:03.835+02:00",
    "start": "2019-05-08T12:16:53.835+02:00"
  },
  "to": "device0.3.1000600",
  "rev": "2-2019-05-08T12:16:50.841+02:00",
  "id": "1423699-1",
  "profile": "10355806-c553-4821-b7e7-f8fd81a1826c",
  "from": "device1.3.90600",
  "createdBy": "api",
  "lockedBy": null,
  "end": 1557310623835,
  "allocationState": 0,
  "cancelTime": null,
  "start": 1557310613835,
  "serviceId": "1423699-1",
  "isRecurrentInstance": false
}
},
```



```
"1423700-1": {
  "formatSubState": 0,
  "legacyType": "p2mp-join",
  "needPersist": true,
  "state": 0,
  "res": {
    "paths": [
      {
        "intervals": null,
        "path": {
          "bidirReversePath": false,
          "main": {
            "edges": [
              {
                "fromNode": "device1.3.90602",
                "toNode": "device1.3.10604.out",
                "id": "device1.3.90602::device1.3.10604.out"
              },
              {
                "fromNode": "device1.3.10604.out",
                "toNode": "device0.3.10604.in",
                "id": "5191ac7d-f279-4b9b-a281-eb81be808ee0"
              },
              {
                "fromNode": "device0.3.10604.in",
                "toNode": "device0.3.1000602",
                "id": "device0.3.10604.in::device0.3.1000602"
              }
            ]
          },
          "status": 0
        },
        "spare": {
          "edges": [
            {
                "fromNode": "device1.3.90602",
                "toNode": "device1.3.10606.out",
                "id": "device1.3.90602::device1.3.10606.out"
            },
            {
                "fromNode": "device1.3.10606.out",
                "toNode": "device0.3.10606.in",
                "id": "10dc0597-8d23-4580-b958-d8a3c17f7d85"
            },
            {
                "fromNode": "device0.3.10606.in",
                "toNode": "device0.3.1000602",
                "id": "device0.3.10606.in::device0.3.1000602"
            }
          ],
          "status": 0
        }
      }
    ]
  },
  "status": 0
},
],
```



```
    "topLevelStatus": null,
    "type": "bookingResult"
  },
  "type": "connection",
  "resultSubState": 0,
  "booking": {
    "lastModifiedBy": "api",
    "parent": null,
    "legacyType": "p2mp-join",
    "templateLink": null,
    "timestamp": 1557310610843,
    "ctype": 2,
    "tags": [
      "tag"
    ],
    "descriptor": {
      "desc": "desc",
      "label": "label"
    },
    "suppressConfig": false,
    "auditHistory": [
      {
        "msg": "New P2MP joined connection created (SRC: device1.3.90602, DST:
device0.3.1000602, ID: 1423700-1). Connection is active from ${timestamp:1557310613}
to ${timestamp:1557310623}",
        "user": "api",
        "rev": "",
        "ts": 1557310610
      }
    ],
    "groupOrder": null,
    "scheduleInfo": {
      "startTimestamp": 1557310613835,
      "endTimestamp": 1557310623835,
      "end": "2019-05-08T12:17:03.835+02:00",
      "start": "2019-05-08T12:16:53.835+02:00"
    },
    "to": "device0.3.1000602",
    "rev": "2-2019-05-08T12:16:50.843+02:00",
    "id": "1423700-1",
    "profile": "10355806-c553-4821-b7e7-f8fd81a1826c",
    "from": "device1.3.90602",
    "createdBy": "api",
    "lockedBy": null,
    "end": 1557310623835,
    "allocationState": 0,
    "cancelTime": null,
    "start": 1557310613835,
    "serviceId": "1423700-1",
    "isRecurrentInstance": false
  }
}
```




Any fields that may be part of a response that is not documented, like "needPersist" and "booking.groupOrder" can be considered deprecated and is most likely only present for internal use and should be removed from REST export at some point in the future.

3.4.5.2 Locking one of the connections (2):

Request

```
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,
    "bookingStrategy": 2,
    "entries": [
      {
        "serviceParameters": {
          "descriptor": {
            "label": "label",
            "desc": "desc"
          },
          "profileId": null,
          "type": "connection",
          "allocationState": 0,
          "tags": [
            "tag",
            "new tag"
          ]
        },
        "genericParameters": {
          "scheduleInfo": {
            "startTimestamp": 1557310613835,
            "type": "once",
            "endTimestamp": 1557310623835
          },
          "locked": true
        },
        "rev": "2-2019-05-08T12:16:50.841+02:00",
        "id": "1423699-1"
      }
    ]
  }
}
```

Response

```
{
```



```
"data": {
  "entriesLink": [
    {
      "error": null,
      "id": "1423699-1"
    }
  ],
  "bookresult": {
    "result": {
      "msg": [],
      "ok": true
    },
    "details": {
      "1423699-1": {
        "availableSpots": [],
        "triedSpotFinding": false,
        "rev": "3-2019-05-08T12:16:50.941+02:00",
        "status": 0,
        "details": [],
        "type": "connection"
      }
    }
  },
  "tmpIdMap": {}
},
"header": {
  "caption": "Operation Successful",
  "id": 1,
  "msg": [
    ""
  ],
  "ok": true,
  "status": "OK"
}
}
```

3.4.5.3 Cancelling both connections with best effort (3):

Request

```
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,
    "bookingStrategy": 2,
    "entries": [
```



```
{
  {
    "rev": "2-2019-05-08T12:16:50.843+02:00",
    "id": "1423700-1"
  },
  {
    "rev": "3-2019-05-08T12:16:50.941+02:00",
    "id": "1423699-1"
  }
]
}
```

Response

```
{
  "data": {
    "entriesLink": [
      {
        "error": null,
        "id": "1423700-1"
      },
      {
        "error": null,
        "id": "1423699-1"
      }
    ],
    "bookresult": {
      "result": {
        "msg": [
          "The service is locked."
        ],
        "ok": false
      },
      "details": {
        "1423699-1": {
          "rev": "3-2019-05-08T12:16:50.941+02:00",
          "status": -26,
          "type": "generic"
        },
        "1423700-1": {
          "availableSpots": [],
          "triedSpotFinding": false,
          "rev": "",
          "status": 0,
          "details": [],
          "type": "connection"
        }
      }
    },
    "tmpIdMap": {}
  },
}
```



```
"header": {
  "caption": "Operation Successful",
  "id": 1,
  "msg": [
    ""
  ],
  "ok": true,
  "status": "OK"
}
```

The currentModernServices map now only has one entry "1423699-1":

```
{
  "currentModernServices": {
    "1423699-1": {...}
  }
}
```

3.4.5.4 Unlocking the last connection (4):

Request

```
{
  "header": {
    "id": 1
  },
  "data": {
    "conflictStrategy": 0,
    "bookingStrategy": 2,
    "entries": [
      {
        "serviceParameters": null,
        "genericParameters": {
          "scheduleInfo": {
            "startTimestamp": 1557310613835,
            "type": "once",
            "endTimestamp": 1557310623835
          },
          "locked": false
        },
        "rev": "3-2019-05-08T12:16:50.941+02:00",
        "id": "1423699-1"
      }
    ]
  }
}
```

**Response**

```
{
  "data": {
    "entriesLink": [
      {
        "error": null,
        "id": "1423699-1"
      }
    ],
    "bookresult": {
      "result": {
        "msg": [],
        "ok": true
      },
      "details": {
        "1423699-1": {
          "availableSpots": [],
          "triedSpotFinding": false,
          "rev": "4-2019-05-08T12:16:50.976+02:00",
          "status": 0,
          "details": [],
          "type": "connection"
        }
      }
    },
    "tmpIdMap": {}
  },
  "header": {
    "caption": "Operation Successful",
    "id": 1,
    "msg": [
      ""
    ],
    "ok": true,
    "status": "OK"
  }
}
```

3.4.5.5 Cancelling the last connection (5):

Request

```
{
  "header": {
    "id": 1
  },
  "data": {
```



```
"conflictStrategy": 0,  
"bookingStrategy": 2,  
"entries": [  
  {  
    "rev": "4-2019-05-08T12:16:50.976+02:00",  
    "id": "1423699-1"  
  }  
]  
}
```

Response

```
{  
  "data": {  
    "entriesLink": [  
      {  
        "error": null,  
        "id": "1423699-1"  
      }  
    ],  
    "bookresult": {  
      "result": {  
        "msg": [],  
        "ok": true  
      },  
      "details": {  
        "1423699-1": {  
          "availableSpots": [],  
          "triedSpotFinding": false,  
          "rev": "",  
          "status": 0,  
          "details": [],  
          "type": "connection"  
        }  
      }  
    },  
    "tmpIdMap": {}  
  },  
  "header": {  
    "caption": "Operation Successful",  
    "id": 1,  
    "msg": [  
      ""  
    ],  
    "ok": true,  
    "status": "OK"  
  }  
}
```

The currentModernServices map is now empty:



```
{
  "currentModernServices": {}
}
```

3.5 Asynchronous Modern Services

```
POST /api/modernServicesAsync
GET /rest/v1/data/status/conman/asyncResponses/**
```

The asynchronous modern services API is commonly referred to as the async API. It is possible to combine the async API with the update API (/api/updateModernServices), but in practice this is rarely necessary as a change is usually implemented by creating a new connection using the async API. The async API also supports cancelling connections directly without using another API.

The purpose of the async API is to facilitate streaming of connection requests from an external system to achieve higher throughput. The async API also allows the VeoIPath system to optimize the routing and configuration pipelines as requests are not synchronously processed by the system.

The async API does not support scheduling. All connections created using the API will be scheduled from now until infinity.

3.5.1 Create connection

This section describes how to create one or more connections using the async API.

Request example:

```
curl --request POST \
  --user api:password \
  --url 'http://veoipath/api/modernServicesAsync' \
  --header 'Content-Type: application/json' \
  --data @requestData.json
```

Request data example:

```
{
  "header": {
    "priority": "normal",
    "id": 1
  },
  "data": {
    "inputs": [
      {
        "connection": {
          "skipReuseJoinId": true,
          "to": "virtual.0.0.2",
          "from": "virtual.0.0.1",
          "allocationState": 0,
          "descriptor": {
```



```
        "desc": "my_description",
        "label": "my_label"
    },
    "profileId": "25ad514e-8a02-4b89-b5da-31e789ef06c3",
    "tags": [
        "my_tag"
    ],
    "type": "connection",
    "ctype": 2
},
"type": "connection"
}
]
```

Request fields:

Field	Type	Values
header	object	The header is identical to that used in set/update/cancel APIs. The request priority may be set to "high", "normal" or "low".
data.inputs	array of object	Each connection object represents one async operation to create or cancel a connection. In this example to create a new connection. The fields have the same meaning as in the set API.

Note that it does not matter if you send requests one by one or chunk these up all at once in a big array. The response is immediate and not transactional like in the case of the set API. The main difference from the set API is that the response only indicates that the request is valid and accepted by the system. Further status about the connection processing is provided asynchronously via the REST API as described below.

Connection object:

Field	Type	Values
to	string	vertex id of the destination/receiver endpoint
profileId	string	ID of the profile associated with the service.
from	string	vertex id of the source/sender endpoint
ctype	int	0 = point to point (unicast), 2 = point to multipoint (multicast), 3 = bidirectional.
allocationState	int	See below.



Field	Type	Values
skipReuseJoinId	boolean	If true, point-to-multipoint services will get service ids the same way as point-to-point (no -1 -2 -3 etc). Recommended for large salvos where setup time is critical.

Allocation state:

Value	Label	Description
0	Normal	Booking is allocated and reserves resources and is configured normally.
1	Suppress config	Booking is allocated and reserves resources but is not configured.
2	No reservation	Booking is allocated but does not reserve resources and is not configured (planned).

Response example:

```
{
  "data": [
    {
      "errors": [],
      "id": "116710-1",
      "pending": true
    }
  ],
  "header": {
    "caption": "Operation Successful",
    "id": 1,
    "msg": [
      ""
    ],
    "ok": true,
    "status": "OK"
  }
}
```

Response fields:

The *header* object is a generic object in all VideonPath responses and can be ignored for the async API. All the relevant information is provided in the *data* object.

Field	Type	Values
errors	array of string	Errors associated with queuing the async request. Note only validation of the request is performed before queuing. Typical error situations include "profile not found" and "source or destination not found".



Field	Type	Values
id	string	Booking ID associated with the async request.
pending	boolean	If true, then the operation is queued and you can expect the response on REST later. If false, then the operation is already complete, either because it failed to be queued or the given service was already provisioned (same source is already connected to the destination with the given profile).

3.5.2 Request Status

This section describes how to retrieve status concerning pending requests made over the async API.

Request example:

```
curl --request GET \  
  --user api:password \  
  --url 'http://videolpath/rest/v1/data/status/conman/asyncResponses/**'
```

Response example:

```
{  
  "_meta" : {  
    "info" : {  
      "auth" : true,  
      "code" : 0,  
      "desc" : "",  
      "internalDetails" : [ ],  
      "msg" : "OK",  
      "ok" : true,  
      "requestDetails" : [ ],  
      "user" : "sysadmin"  
    },  
    "type" : "mainResult"  
  },  
  "data" : {  
    "status" : {  
      "conman" : {  
        "asyncResponses" : {  
          "116710-1" : 8  
        }  
      }  
    }  
  }  
}
```

The *header* object is a generic object in all VideolPath responses and can be ignored. All the relevant information is provided in the *data* object.



All responses to pending async requests will be posted on `/rest/v1/data/status/conman/asyncResponses` as a map from booking ID (obtained from the async response with `pending=true`) to a status code.

A response will be available for 10 seconds. The response will be logged in the system log, so the information is not lost forever, just not available directly to the client to avoid overload in the event a lot of services are repeatedly requested.

Typically you want to observe changes to the async responses to ensure that requests are fulfilled without errors. Alternatively you can use the async api as a more fire and forget style. Typically these are the cases you end up in:

1. Response code 8 (already ok): This results from `pending=false` with zero errors and can be handled in the client response directly.
2. Other positive codes: This results from `pending=true` with zero errors, and ends in a REST update and could be handled directly from there.
3. Negative codes (errors): This results from a booking error (e.g. no path). This will not result in a REST update and should be handled by the client. A negative code means that the connection is not created.

This leads to two primary paths to select:

1. Handle all responses from the async responses API as well as booking updates from the REST API.
2. Handle timeouts in the client and `pending=true` requests that doesn't respond in a given time mark as suspicious and potentially an error.

For a complete list of status codes please refer to the general modern services page. However, in most cases it is sufficient to treat positive codes as success and negative codes as error.

Response example for failure:

```
{
  "data": [
    {
      "errors": [
        "Parameter error: Profile with id '25ad514e-8a02-4b89-b5da-31e789ef06sc3' not found."
      ],
      "id": "",
      "pending": false
    }
  ],
  "header": {
    "caption": "Operation Successful",
    "id": 1,
    "msg": [
      ""
    ],
    "ok": true,
    "status": "OK"
  }
}
```

Note that the `id` field is empty in this case as no connection has been queued due to the error. Also `pending=false` so no further status update will be posted on the REST API.



3.5.3 Cancel Connection

This section describes how to cancel one or more connections using the async API.

Request example:

```
curl --request POST \  
  --user api:password \  
  --url 'http://videoipath/api/modernServicesAsync' \  
  --header 'Content-Type: application/json' \  
  --data @requestData.json
```

Request data:

```
{  
  "header": {  
    "priority": "normal",  
    "id": 1  
  },  
  "data": {  
    "inputs": [  
      {  
        "type": "cancel",  
        "rev": "2-2020-08-31T12:40:17.725+02:00",  
        "id": "116710-1"  
      }  
    ]  
  }  
}
```

When canceling a connection using the async API it is only required to provide the *id* and *rev* (revision) of the connection. Note that the *type* field is set to "cancel" in this case.

Response example:

```
{  
  "data": [  
    {  
      "errors": [],  
      "id": "116710-1",  
      "pending": true  
    }  
  ],  
  "header": {  
    "caption": "Operation Successful",  
    "id": 1,  
    "msg": [  
      ""  
    ]  
  },  
  "ok": true,
```



```
"status": "OK"
}
```

The format of the response is the same as for the create connection case above.

3.6 Group connection API

⚠ This API is available in VideolPath v11 and is considered experimental at this stage

3.6.1 Retrieve Group Resources

```
GET /rest/v1/data/status/conman/endpoints/*%20where%20specific.type='group' /
**

GET /rest/v1/data/status/conman/endpoints/*%20where%20specific.type='group' /
generic/alias/*/.../.../descriptor/*/.../.../endpointType,id,universal/.../
tags/*
```

The first request returns all groups and all details of these groups. This request does not apply any filtering and returns a large volume of data therefore it should not be used for any integrations.

The second request returns all groups currently in the system that are accessible to the user. It includes an elaborate filter that has been designed to retrieve the most relevant information related to groups. The request returns groups together with information about aliases, labels, description, group type in/out, ID and tags. It is recommended to use this request to retrieve group information

The third request below returns additional information about the inputs and outputs within the group. This information is not required for making group connections

```
GET /rest/v1/data/status/conman/endpoints/*%20where%20specific.type='group'/generic/
alias/*/.../.../descriptor/*/.../.../endpointType,id,universal/.../tags/*/.../.../...
/specific/destEndpoints,sourceEndpoints/*/endpoint/generic/
alias,endpointType,id,tags/*/.../.../**
```

3.6.1.1 Request Example

```
curl --request GET \
  --user api:password \
  --url 'http://videopath/rest/v1/data/status/conman/endpoints/
*%20where%20specific.type='group'/generic/alias/*/.../.../descriptor/*/.../.../
endpointType,id,universal/.../tags/*1'
```

¹ http://videopath/rest/v1/data/status/collector/virtualResources/*/descriptor/desc,label/.../.../props/hasInputs,hasOutput/*



3.6.1.2 Response Example

This example below includes data typically provided for the second request above.

```
"data": {
  "status": {
    "conman": {
      "endpoints": {
        "grp:051474ba-d201-46c2-9297-8d3821c6c4fe": {
          "generic": {
            "alias": {
              "a0": "UDX02",
              "a1": "UDX02",
              "a2": "UDX02",
              "a3": "UDX02"
            },
            "descriptor": {
              "desc": "",
              "label": "UDX02"
            },
            "endpointType": "Out",
            "id": "grp:051474ba-d201-46c2-9297-8d3821c6c4fe",
            "tags": [
              "2110-30 6ch",
              "#UDX",
              "2110-40",
              "2110-30 2ch",
              "2110-30 4ch",
              "2110-20"
            ],
            "universal": false
          }
        }
      }
    }
  }
}
```

3.6.1.3 Response Fields

Field	Type	Description
:generic	object{}	Generic group information
:generic.id	string	Identity of the group
:generic.descriptor.label	string	The label associated with the group
:generic.descriptor.desc	string	Description associated with the group
:generic.alias.a[]	string	alias name defined for a group, if not set then default label is used



Field	Type	Description
:generic.endpointType	string	'Out' - receiver group, 'In' - sender group
:generic.tags	string[]	An array of tags associated with the group (for filtering purposes)
:generic.universal	boolean	

3.6.2 Connect Group Resources

POST /api/connect

This request is used to connect groups. Groups contain input or output endpoints. When two groups are connected the system tries to find a match between inputs and outputs using content-ID's assigned to endpoints. The number of endpoints defined in the IN and OUT groups does not have to be the same. The system will connect only endpoints with matching content-ID's. The association between profiles and content-ID's are defined in the group schemas. Schemas are part of VideolPath configuration and are not relevant for group connection provisioning.

3.6.2.1 Request Example

```
curl --request POST \  
  --user api:password \  
  --url 'http://videopath/api/connect' \  
  --header 'Content-Type: application/json' \  
  --data @requestData.json
```

3.6.2.2 Request Data Example

Request

```
"data": {  
  "entries": [{  
    "from": "grp:f4c9b785-a150-4a06-8a3c-3ae73b4568d0",  
    "profiles": [],  
    "tags": [],  
    "to": "grp:602f2e22-be44-4937-9036-e399091b1195",  
    "scheduleInfo": {  
      "type": "once",  
      "startTimestamp": null,  
      "endTimestamp": null  
    },  
    "ctype": "p2mp"  
  }  
}
```



```
    }  
  ],  
  "bookingStrategy": 2,  
  "conflictStrategy": 1  
}
```

3.6.2.3 Request Fields

Field	Type	Description
entries	object[]	An array of group connection entries
entries[].descriptor.label	string	The label associated with the group connection
entries[].descriptor.desc	string	Description associated with the group connection
entries[].from	string	The ID of the 'from' group
entries[].scheduleInfo	object	The schedule information objects as described under Set Modern Services (see page 31). If the "startTimestamp" and "endTimestamp" are set to null the connection starts immediately and does not end automatically. Needs to be cancelled.
entries[].tags	string[]	An array of tags associated with the group connection
entries[].to	string	The ID of the 'to' group
entries[].profiles	string[]	An array of profile IDs is requested to be used. May be empty if there is no request.
entries[].ctype	string	Connection type: "p2mp"- point-to-multipoint - this is a default 'ctype' for multicast connection. The "p2p" - point-to-point connection type is used for unicast services
bookingStrategy	integer	Booking strategy as described under General Request Structure (see page 25)
conflictStrategy	integer	Conflict strategy as described under General Request Structure (see page 25)



3.6.2.4 Response Example

```
{
  "data": {
    "entries": [
      {
        "id": "grp_702b1ef6-58f5-43b3-a9e4-c46ad6204d3f-c08e63d0-
d051-467f-9677-e025cc133ba7-1660644224757",
        "result": {
          "msg": [],
          "ok": true
        }
      }
    ],
    "header": {
      "id": 0,
      "msg": [],
      "ok": true,
      "status": "OK"
    }
  }
}
```

3.6.2.5 Response Fields

Field	Type	Description
entries	object[]	An array of group connection results
entries[].id	string	Group ID allocated to the connection
entries[].result.msg	string[]	An array of messages describing the problem if a connection is not successful
entries[].result.ok	boolean	True if the group connection was successful

3.6.3 Retrieve Group Connections

```
GET /rest/v1/data/status/conman/services/*%20where%20type='group'/connection/
connection.generic,generic/**/.../.../connection.to,from,to,id,rev,specific/
specific.breakAway,breakAway,complete,missingActiveConnections,numChildren,children/*
```



3.6.3.1 Request Example

```
curl --request GET \  
  --user api:password \  
  --url 'http://videoipath/rest/v1/data/status/conman/services/  
*%20where%20type='group'/connection/connection.generic,generic/**/.../.../  
2connection.to3,from,to,id,rev,specific/  
specific.breakAway,breakAway,complete,missingActiveConnections,numChildren,children/*4'
```

3.6.3.2 Response Example

This example below includes data typically provided for the first request above.

```
"data": {  
  "status": {  
    "conman": {  
      "services": {  
        "grp_24beb99f-881b-4f3d-97c6-373ba4a4b24c-67f84419-e342-4aac-83bf-  
d100aaba9550-1660646267325": {  
          "connection": {  
            "from": "grp:24beb99f-881b-4f3d-97c6-373ba4a4b24c",  
            "generic": {  
              "descriptor": {  
                "desc": "",  
                "label": ""  
              },  
              "locked": false,  
              "state": 1  
            },  
            "id": "grp_24beb99f-881b-4f3d-97c6-373ba4a4b24c-67f84419-  
e342-4aac-83bf-d100aaba9550-1660646267325",  
            "rev": "79114430-1d4f-11ed-9b15-e1701b67b108",  
            "specific": {  
              "breakAway": false,  
              "children": {  
                "101946-1": {},  
                "101947-1": {},  
                "101948-1": {},  
                "101949-1": {},  
                "101950-1": {}  
              },  
              "complete": true,  
              "missingActiveConnections": {},  
              "numChildren": 5  
            },  
            "to": "grp:67f84419-e342-4aac-83bf-d100aaba9550"
```

² http://videoipath/rest/v1/data/status/pathman/currentModernServices/* where type=%27group%27/**

³ <http://connection.to>

⁴ http://videoipath/rest/v1/data/status/pathman/currentModernServices/* where type=%27group%27/**



```
    },  
    "grp_d41fb80a-aa3c-4f8b-a9ef-17168b176f25-67f84419-e342-4aac-83bf-  
d100aaba9550-1660646267325": { ...
```

3.6.3.3 Response Fields

Field	Type	Description
:id.connection.from	number	ID of the 'from' group
:id.connection.generic	object	generic connection information
:id.connection.generic.descriptor	object	Descriptor object with label and desc string fields.
:id.connection.generic.locked	boolean	Locked status false/true
:id.connection.generic.state	int	0- Scheduled State, "Booking is valid and scheduled" 1- Active State, "Connection is active now" 2- Ended State, "Connection is ended" 3- Canceled State, "Connection is cancelled by the end user"
:id.connection.id	string	connection ID
:id.connection.rev	string	Revision of the connection. Must be specified when making changes.
:id.connection.specific	object	The specific object includes group connection-specific information
:id.connection.specific.breakAway	boolean	Group connection break away status: true/false
:id.connection.specific.children	object	The children object includes a list of single connection IDs that are part of the connection group.



Field	Type	Description
:id.connection.specific.complete	boolean	If set to 'true' it indicates that all children connections are part of the group throughout the whole connection duration. If set to 'false' it indicates that the group booking at some point is missing some children connection (not all content-id's defined in the group schema have active connections). It may have connections for all content Ids during parts of the booking period, but not during the whole period, or it might be missing connections during the whole booking period
:id.connection.specific.missingActiveConnections	object	An object that includes a list of content-id's from the group schema that was not completed during group connection. If the group is complete: true then this object is empty
:id.connection.specific.numChildren	int	A number of children (single connections) that are part of the group
:id.connection.to	string	ID of the 'to' group

3.6.4 Update Group Connections

POST /api/update

Update group connections are performed using the update API

3.6.4.1 Request Data Example

The example request below shows how to lock a group connection.

Request

```
{
  "header": {
    "id": 0
  },
  "data": {
    "entries": [{
      "id": "grp_24beb99f-881b-4f3d-97c6-373ba4a4b24c-67f84419-e342-4aac-83bf-d100aaba9550-1660646267325",
      "rev": "5796ca23-1e6a-11ed-9b15-e1701b67b108",
    }]
```



```
        "locked": true
      },
    ],
    "bookingStrategy": 2,
    "conflictStrategy": 1
  }
}
```

3.6.5 Cancel Group Connections

POST /api/disconnect

Cancel group connections is performed using the disconnect API

3.6.5.1 Request Data Example

The example request below shows how to cancel a group connection.

Request

```
{
  "header": {
    "id": 0
  },
  "data": {
    "entries": [{
      "id": "grp_702b1ef6-58f5-43b3-a9e4-c46ad6204d3f-c08e63d0-
d051-467f-9677-e025cc133ba7-1660768018345",
      "rev": "f218ed83-1e6a-11ed-9b15-e1701b67b108"
    }
  ],
  "bookingStrategy": 3,
  "conflictStrategy": 1
}
```



4 Federation

Federation is a way to connect multiple systems together and to be able to establish connections between the systems. Each system is responsible for its local resources and can operate completely autonomously in case communication between the systems is broken.

There are two different federation models:

- Public endpoints. In this case endpoints from a system may be marked as public and exposed to another system permanently as external endpoints.
- Handoff point. In this case any local source may be routed to a handoff point between two systems. The source will temporarily be exposed to the other system for the duration of the connection in the originating system.

4.1 External endpoints

```
GET /rest/v1/data/status/network/externalEndpoints/*/
useAsEndpoint,codecFormat,vertexType,type,descriptor/desc,label/**
```

Get list of external endpoints that are exposed by other connected federated systems. Public endpoints from other systems will always be available here, while local endpoints from other systems will appear here when they are sent to the handoff point between the systems.

The data is filtered by the ID, name, description, tags and type/equipment properties associated with the endpoint.

4.1.1 Example Request

```
curl --request GET \
  --user api:password \
  --url 'http://videoipath/rest/v1/data/status/network/externalEndpoints/*/
useAsEndpoint,codecFormat,vertexType,type,descriptor/desc,label/**'
```

4.1.2 Example Response

```
{
  "_meta": {
    ...
  },
  "data": {
    "status": {
      "network": {
        "externalEndpoints": {
          "|-federated-virtual.1.0.3": {
            "codecFormat": "Video",
            "descriptor": {
              "desc": "",
              "label": "INPUT"
            }
          }
        }
      }
    }
  }
}
```



```
    },
    "type": "codecVertex",
    "useAsEndpoint": true,
    "vertexType": "In"
  }
}
}
```

4.1.3 Response Data

The ID for external endpoints has the following format:

|<external_system_id>-|<external_system_endpoint_id>

Example:

|<federated>-|virtual.1.0.3>

Field	Type	Values
descriptor	object	
desc	string	Description of the endpoint (from external system)
label	string	Label (name) of the endpoint (from external system)
type	string	Type of vertex. Typically generic or codecVertex for an endpoint. Remaining fields may vary depending on type.
codecFormat	string	Possible values "Video", "Audio" or "ASI". Only present for codec vertices.
vertexType	string	Possible values "In", "Out" or "BiDirectional"
useAsEndpoint	boolean	True if this vertex is used as endpoint
tags	array of string	Tags associated with the endpoint



4.2 Handoff points

```
GET /rest/v1/data/status/network/externalTopologyInfo/**
```

Get list of handoff points towards other external systems. These may be used to establish connections from local sources towards other systems.

4.2.1 Example Request

```
curl --request GET \
  --user api:password \
  --url 'http://videoipath/rest/v1/data/status/network/externalTopologyInfo/**'
```

4.2.2 Example Response

```
{
  "_meta": {
    ...
  },
  "data": {
    "status": {
      "network": {
        "externalTopologyInfo": {
          "federated": {
            "handoffId": "|-federated-|PrivateTopology.HandoffPoint"
          }
        }
      }
    }
  }
}
```

4.2.3 Response Data

Field	Type	Values
externalTopologyInfo.<id>	string	ID of the external systems that this is a handoff point for (e.g. federated as shown above).
externalTopologyInfo.<id>.handoffId	string	ID of the handoff point towards the external system (this should be used in connections from local sources as destination ID).



The ID for handoff points has the following format:

```
|-<external_system_id>-|<external_system_handoff_point_id>
```

Example:

```
|-federated-|PrivateTopology.HandoffPoint
```

4.3 Connections

In order to make connections it is enough to utilize standard Modern Services API and use external endpoint ID as a sender or a receiver e.g `|-federated-|device3.1.1000002` .

For connections between public endpoints, the connection will be successful only between endpoints that are public. Public endpoints may be retrieved as follows:

- External endpoints may be retrieved using `/rest/v1/data/status/network/externalEndpoints` as described above.
- Local public endpoints may be retrieved using `/rest/v1/data/config/network/nGraphElements/*` where `value.useAsEndpoint=true` and `value.public=true` . This filters all endpoints that have the public flag set to true.

Note that this does not apply for connections to handoff points. In this case local sources may be routed to the handoff point and this will appear as an external endpoint in the other system.